

Decision brief: the settled rulings revisited, and the open choices with their pros and cons

- **Status:** FOR REVIEW AND ADJUDICATION. Written as a handoff: other reviewers (people or models) can mark it without the repository. Nothing here changes a ruling. Where it recommends amending one, the owner decides.
- **Revision:** r4.4, 2026-09-30, by Claude.
- **Asked:** the owner asked, "Are any of the closed rulings worth revisiting? Can you provide a handoff document that reviews the choices and recommendations to make, along with pros and cons."
- **Companion documents:**
 - HANDOFF-REVIEW-PACKAGE.md: what the pattern is, Packages A to C and their registers.
 - LTC-INTEGRATION-REVIEW.md (Package A).
 - BREAK-GLASS.md (Package B).
 - PD-01-DAEMON-CLASSES.md.

How to read the verdicts. A settled ruling can be kept, kept with an amendment (the decision stands; how it is built or worded changes), clarified (the decision stands; its scope is stated), or reversed. Every finding says what kind of evidence stands behind it:

- *measured:* run and recorded;
- *traced:* read in the code;
- *reasoned:* argued, not run.

1. Answer in brief

No settled ruling should be reversed. Each has held up, and one was confirmed by a new measurement. Each of the four does have one amendment or clarification worth making, and two of them fix real weaknesses in how the ruling was built:

Ruling	Verdict	What is worth revisiting	Evidence	Urgency
PD-63 Blind period	Keep, amend how it was built	SENSE_BLIND and a policy violation share exit code 78. When the output directory is unsafe, the ledger cannot be written and the exit code is the only signal left, so a security event looks like an operational one	Traced (spark_daemon/__init__.py: EXIT_POLICY = 78, EXIT_SENSE_BLIND = 78)	Medium; batch with the exit-code work (PD-35, C-1)
PD-70 Blindness survives restarts	Keep, amend how it was built	Blindness inherited across a restart is measured on the wall clock. If the clock steps back, inherited blindness reads as zero, and the slow restart loop that PD-70 fixed (HF-32) hides again. If it steps forward, a healthy daemon is judged blind	Traced (_seconds_since clamps a future timestamp to 0)	Medium-high; small and additive
PD-01.9 Blind Forester	Keep, clarify its scope	"Critical downstream payload" can be read as the critical consequence level, which PD-01.2 refuses until a certification path exists. The ruling should say that it does not unlock critical, and that the far side's own procedure (BF-5) is mandatory, not recommended	Reasoned	High, before any active class is designed

Ruling	Verdict	What is worth revisiting	Evidence	Urgency
PD-76 Direct memory reading	Keep; amend the mechanism's wording	The ruling names cgroup v2's <code>memory.peak</code> from a <code>systemd</code> transient scope. This workspace has neither, yet a direct reading <i>can</i> be taken here, from a directly created cgroup (v1). The measurement confirms the ruling's premise: DB-14's basis read 128 MiB where the cgroup peaked at 208 MiB	Measured (evidence/r4.4/)	Medium; it removes most of the "INCOMPLETE in CI" problem

The r4.1 statement that "this workspace cannot take the reading" was wrong, and it is corrected where it appeared (RECONCILIATION-V01-LINE.md §3, the decision log).

2. The settled rulings, one by one

2.1 PD-63: the blind period (settled 2026-09-29)

Decided: a daemon that accepts no observation for `T (blind_limit_seconds)` stops with `SENSE_BLIND`, exit 78, and the unit never restarts it. `T` is declared in the manifest.

Learned since:

- **The exit code is shared.** `EXIT_POLICY` (an unsafe output directory, or a purity or policy violation: possibly an attack) and `EXIT_SENSE_BLIND` (the daemon cannot see: operational) are both 78. The ledger distinguishes them, but the unsafe-output-directory case is exactly the one where the ledger cannot be written, so the exit status is all that `systemd`, the journal and an `onFailure=` notifier see. The two need different people at different urgency.
- **Why 78 was chosen:** the Observer's own table uses 78 for `SENSE_BLIND` (WBS 3.1), and this pattern aligns with the Observer voluntarily (PD-35).
- **A consequence of HF-33:** with a persistently over-capacity folder, `dir-watch` now fails every cycle and stops at `T`. That is the ruling working as intended: a capacity problem becomes a visible stop, not a silently wrong inventory. It is stated here so that it is chosen knowingly.

Options for the exit code:

Option	For	Against
A. Keep both at 78	No change; matches the Observer for <code>SENSE_BLIND</code>	A security stop and an operational stop are indistinguishable without the ledger, and the ledger may be the thing that failed
B. Move <code>EXIT_POLICY</code> to its own code; keep <code>SENSE_BLIND</code> at 78 (recommended)	Keeps the Observer alignment for the code the Observer defines. Separates security from operations where it matters most. <code>RestartPreventExitStatus</code> simply lists both	A contract major (exit codes are in the contract). Batch it with PD-35 and C-1
C. Move <code>SENSE_BLIND</code> instead	—	Breaks the Observer alignment

Options for where `T` is set (a lower priority):

Option	For	Against
A. In the manifest (as ruled; recommended for now)	Hashed into <code>DAEMON_START</code> and bound to activation; changing it is visible	Deploying the same daemon on two hosts with different <code>T</code> needs two manifests

Option	For	Against
B. The manifest declares a range; the activation register (PD-40) picks T within it and records the choice	Per-deployment tuning without re-approving the daemon	Needs the register, which is not built. Two places to look for one number

Recommendation: keep PD-63. Adopt exit-code option B in the batched contract change. Revisit where T is set when PD-40 is built.

2.2 PD-70: blindness survives restarts (settled 2026-09-30)

Decided:

- a heartbeat every $T/2$;
- at start-up, the blind clock begins at the ledger's latest evidence of an accepted cycle;
- one reacquisition cycle per restart, so that no daemon is locked out.

Learned since: the time since that evidence is measured on the **wall clock** (`_seconds_since`, which returns 0 when the timestamp lies in the future). Two failure cases follow from the code:

- **The clock steps back** (an NTP correction after a fast real-time clock, or a manual change). The last accepted timestamp then lies "in the future", inherited blindness reads as 0, and every restart starts a fresh countdown. This brings HF-32 back: a slow restart loop hides blindness indefinitely, which is the silent outage PD-70 was adopted to prevent.
- **The clock steps forward** (a real-time clock reset at boot, then NTP). A healthy daemon inherits a large blindness and gets only its single reacquisition cycle. That is safe, but it is a false stop.

Within one boot there is a clock that no wall-clock change moves: Linux's `CLOCK_BOOTTIME`, which is shared by all processes since boot and counts suspended time. Across a reboot no local clock is reliable. The Observer's own rule (D-7) already keeps monotonic time within a process.

Option	For	Against
A. Keep the wall clock (as built)	Simple	A backward step silently brings back the defect PD-70 fixed
B. Record <code>boot_id</code> and <code>CLOCK_BOOTTIME</code> with every piece of acceptance evidence (heartbeat, start, stop). Within the same boot, measure on <code>CLOCK_BOOTTIME</code>. Across boots, use the wall clock; if it went backward, or cannot be trusted, assume the worst (blindness at the limit, so one reacquisition cycle). Record <code>clock_basis</code> (recommended)	Immune to clock steps within a boot, which is where restart loops happen. Honest across boots (S-8), and errs towards a check, never towards silence	Additive fields in reserved records (a contract minor). Blindness measured from the last heartbeat instead of the last event can over-state it by up to $T/2$, in the safe direction
C. Always assume the worst across restarts	Simplest safe rule	Every restart costs its one reacquisition cycle; a daemon on a flaky input stops more often

Recommendation: keep PD-70 and build option B. A test can inject a backward step through the ledger's timestamps; it should fail on the current code and pass after the change.

2.3 PD-01.9: the "Blind Forester" protocol (settled 2026-09-30, adopted as core pattern)

Decided: an active daemon whose stopping would drop a critical downstream payload does not fail closed when blind. It shifts into a pre-validated, degraded survival loop and signals distress. A restart resumes that loop. For observers the survival action stays "fail closed". The ruling superseded, in part, the earlier boundary that no DGX process may sit in a loop that keeps a person safe.

Learned since:

1. **The word "critical" is ambiguous.** It can be read as the consequence level `critical`, which PD-01.2 refuses "until a certification path exists". Read that way, the ruling would unlock life-safety duty for a general-purpose AI host with no certification. That is almost certainly not intended, but the text does not rule it out.
2. **BF-5 (the far side keeps its own procedure) is still only a recommendation.** It is the condition that keeps a person safe if the whole DGX fails: its kernel, power or disk.
3. **Break Glass (Package B) separates survival *mode* from emergency *authority*.** The mode continues across a restart; the authority does not. The Blind Forester ruling should say which one it covers: the mode.
4. **Exit from survival mode.** BF-6 requires revalidation. PD-01.14 proposes that the operator-gated recovery path be the only exit.
5. **Unit rules.** Today the unit never restarts exit 78. An active daemon that exits 78 when blind would drop its payload, the opposite of the ruling. BF-1 (the loop runs in the supervisor, which does not exit) resolves this, so the supervisor split (PD-72) is a hard prerequisite, not an option.

Option	For	Against
A. Keep as ruled; conditions pending	No change	The scope ambiguity stays open until someone designs against it
B. Keep, and add a scope clarification (recommended): the protocol applies to active classes at standard and elevated consequence; it does not unlock <code>critical</code> , which stays refused under PD-01.2; BF-5 is mandatory wherever a person could be harmed; it governs survival <i>mode</i> , never emergency <i>authority</i> ; the supervisor split is a prerequisite	Keeps the owner's intent. Closes the reading that would put an uncertified host in a life-safety loop. Aligns with Package B	The owner may have intended <code>critical</code> to be reachable. If so, that needs its own decision with a certification path
C. Narrow the ruling to elevated only, or return to the old boundary	The simplest safety case	Loses a first line of defence the owner deliberately adopted

Recommendation: keep PD-01.9 with the clarification in option B, and confirm BF-1 to BF-6, with BF-5 as mandatory.

2.4 PD-76 (= the v0.1 line's PD-24(a)): a direct memory reading (settled 2026-09-29 in that line)

Decided: the memory check needs a direct reading of `cgroup memory.peak` from a fresh transient scope containing the daemon and every child. A process-RSS reading is a proxy that never decides, and an RSS-only run is INCOMPLETE.

Learned since (measured, evidence/r4.4/). Running as root on this workspace, with no systemd user manager, a child memory cgroup was created under the shell's own cgroup. A process was moved into it that held 80 MiB while its child allocated 120 MiB:

Basis	Reading
DB-14 today: <code>max(daemon, largest child)</code>	128 MiB
<code>cgroup v1 memory.max_usage_in_bytes</code> , daemon plus every child	208 MiB

- **The premise is confirmed.** DB-14's basis reads about 38 % low whenever a daemon holds memory while a child runs, and `MemoryMax=` enforces the cgroup figure.
- **The mechanism in the ruling is narrower than necessary.** This workspace uses cgroup v1 (no `memory.peak`) and has no systemd user manager (no transient scope), yet a direct reading was taken. So the r4.1 conclusion, that every battery run here would be INCOMPLETE, was wrong.

Option	For	Against
A. Keep the wording (v2 memory . peak from a systemd transient scope)	Exactly what the target, the DGX on cgroup v2 with systemd, will use	INCOMPLETE on every host without both, including this one, although a direct reading is possible
B. Keep the ruling; widen the mechanism (recommended): "a direct cgroup peak for a fresh cgroup holding the daemon and every child: v2 memory . peak, or v1 memory . max_usage_in_bytes, created by a systemd transient scope or directly where the battery may create cgroups. The report records the cgroup version and how the cgroup was created. INCOMPLETE only where no direct reading is possible"	Same substance. INCOMPLETE becomes rare instead of universal, and CI can require PASS where it runs as root or with delegated cgroups	v1 and v2 account for memory differently (page cache, kernel memory), so readings from different versions are not interchangeable. Compare each only against a limit on the same kind of host. The battery needs cgroup write access
C. Go back to RSS	—	Measured 38 % low; reversing would contradict the evidence

Recommendation: keep PD-76 and adopt option B's wording. Build DB-19 on it, and record the cgroup version in the report.

3. The open decisions: choices, pros and cons

Each row gives the realistic options. The recommended option is marked **(rec.)**.

3.1 Package A: the Local Transport Contract

Decision	Options	Pros	Cons
PD-82 Where the LTC sits	Bind at declared connections (rec.) / embed in the pattern / keep separate	Bind: one transport meaning, observers untouched, the gate's tests <i>are</i> the LTC suite	Bind: the seam is designed before its first use. Embed: couples version lines and pulls daemon constants into the LTC. Separate: each class re-derives transport rules
PD-83 Truncated reads	The framework raises TooLarge by default; partial results only by opt-in (rec.) / a battery check only / leave it to authors	Framework: closes the API shape behind HF-29 and HF-33 for every author	Framework: a contract major. Battery-only: catches only what a fixture exercises. Authors: two defects already show this fails
PD-84 Register vocabulary	Adopt the LTC binding fields, with H02's modifications (rec.) / keep this pattern's own names	One vocabulary for daemons and providers	Ties the register's design to an unadjudicated LTC; mitigated because the fields are semantic, not a wire format
PD-85 Pacing	Declared field; deterministic spread by default (rec.) / keep random jitter (PD-21) / none	Deterministic: reproducible and testable; fixed-phase workloads get no hidden jitter	Deterministic: a fixed phase per daemon, so two daemons that hash close together stay close (mitigated by also hashing the host identity). Random: not reproducible
PD-86 Retry bounds	Bounded over time, checked across layers (rec.) / per-layer bounds only	Closes the HF-28 class of loop	Every retrying layer must declare a lifetime bound or a terminal state

3.2 Package B: Break Glass and recovery

Decision	Options	Pros	Cons
PD-01.10 Tiers	Beacon first, acting with class 3b (rec.) / build both together / defer all	Beacon first: useful early, cannot act, proves the recovery states before anything depends on them	Beacon first: two build stages. Both together: waits for kernel v2. Defer: no distress signal when the control plane is lost

Decision	Options	Pros	Cons
PD-01.11 Egress relay	Always present and dormant (rec.) / started only on a trip	Dormant: nothing privileged has to start a unit at the worst moment	Dormant: a standing egress permission, although policy-gated and narrowly scoped. On-trip: no standing egress, but needs a privileged starter
PD-01.11 Survival socket	The owner's revised socket (rec.) / no socket (physical console only) / the original unauthenticated socket	Revised: zero friction for declared peers; exists only during an emergency	No socket: the smallest attack surface, but no local controller path. Unauthenticated: rejected, because it can be squatted and contradicts S-5
PD-01.12 Restart during an emergency	Break glass ends and survival continues (rec.) / allow re-arming with old envelopes	Conservative: authority never outlives its epoch	Conservative: after a crash, only the far side's procedure protects the payload. Permissive: stolen or stale envelopes live longer
PD-01.13 Witness	Separate hardware where a person could be harmed; a separate process tree otherwise (rec.) / always same-host / always separate hardware	Matches independence to consequence (U-7, BM-3)	Separate hardware costs money and needs integration. Same-host shares failure modes
PD-01.14 Recovery gate	Operator required at critical, declared per deployment below that (rec.) / always operator / always automatic	A human boundary where it matters; automatic where a stop costs only time	Always-operator: slow recovery for trivial daemons. Always-automatic: broad authority returns because a heartbeat did

3.3 Package C: other decisions that gate the next steps

Decision	Options	Pros	Cons
BF-1 to BF-6	Confirm all, BF-5 mandatory (rec.) / confirm a subset	Makes the adopted ruling buildable and safe	None material; each condition follows from a recorded invariant
PD-72 Supervisor/worker split	A precondition for any active class; build with the first one (rec.) / build now / never	Required by BF-1 and LTC-14; building it later avoids premature complexity	Build now: effort with no active class to use it. Never: the Blind Forester cannot be built
PD-01.1 to PD-01.3 Classes, consequence field, Sentinel	Approve; ship 1b next (rec.) / keep 1a only	1b closes silent outages at the source; a consequence field makes risk explicit	A contract change; alert fatigue unless ISA-18.2's rules (A-5) come with it
PD-69 Limits checked against each other	Approve (rec.) / ad hoc	Four defects came from limits checked alone (HF-28, HF-30, HF-31, HF-32)	A budget table to maintain
PD-71 (remainder) Worst-case fixtures, N_max	Approve with DB-19 (rec.) / defer	Evidence where the risk is, not on one small fixture	More battery time
PD-75 to PD-81 Numbering	Alias (rec.) / renumber this repository	Alias: no document rewritten	Renumber: churn across many documents; either works if chosen once
C-1 Battery exit codes	Adopt 0/3/4 (PASS/FAIL/INCOMPLETE) in the batched change (rec.) / keep 0/1/3 / map	One convention with SPS-1	A breaking change for anyone scripting on

Decision	Options	Pros	Cons
			today's codes; batch it once
INCOMPLETE in CI	Require PASS where a direct reading is possible; allow INCOMPLETE with a stated environment reason elsewhere (rec.) / always require PASS / always allow	Honest, and after §2.4 rarely needed	Always PASS: fails on hosts that cannot measure. Always allow: hides real gaps
Phase 1 (OnFailure=notifier, diverse staleness checker, budget report)	Go (rec.) / wait	No contract change; each item closes a monitoring gap	Small effort

4. Suggested order of rulings

1. **Clarify PD-01.9** (§2.3). It frames everything designed for active classes.
2. **Confirm BF-1 to BF-6 and the Phase 1 go-ahead.** Neither changes the contract.
3. **Amend PD-70's clock basis** (§2.2). Small, additive, and it closes a regression path for an owner ruling.
4. **Adopt PD-76's wider mechanism** (§2.4), and with it the CI rule for INCOMPLETE. Then build DB-19.
5. **The batched contract change (4.0.0):**
 - PD-63's exit-code split;
 - C-1 and PD-81 (battery exit codes);
 - PD-35, PD-39 and PD-49;
 - PD-83 (truncated reads);
 - PD-85 (pacing);
 - PD-01.2 (consequence field).
6. **PD-82, PD-84 and PD-86**, as design rules for the register and the gate.
7. **Package B** (PD-01.10 to PD-01.14), after PD-72 and PD-01.7 are ruled on.

5. Markup template

Item	Verdict (KEEP / AMEND / CLARIFY / REVERSE, or APPROVE / MODIFY / REJECT / DEFER)	Evidence type	Notes
PD-63: exit-code split			
PD-63: where T is set			
PD-70: clock basis			
PD-01.9: scope clarification			
PD-76: mechanism wording			
PD-82 to PD-86			
PD-01.10 to PD-01.14			
BF-1 to BF-6			
PD-72			
PD-01.1 to PD-01.3			
PD-69, PD-71			
PD-75 to PD-81, C-1			
INCOMPLETE in CI			

Item	Verdict (KEEP / AMEND / CLARIFY / REVERSE, or APPROVE / MODIFY / REJECT / DEFER)	Evidence type	Notes
Phase 1			